Roteiro de Apresentação — Entrega 2

Plataforma de Estágios — UNIPI Conecta

Objetivo da apresentação

Apresentar o sistema funcionando pelo site e explicar, de forma objetiva, a arquitetura em camadas do backend e a comunicação síncrona entre a aplicação Django e o microserviço.

Estratégia

Não é necessário ficar apresentando código por 15 minutos. A ideia é usar o site como parte principal da demonstração e mostrar apenas os arquivos necessários para comprovar a arquitetura.

Etapa	Tempo	O que mostrar
1. Abertura	1 min	Site e objetivo do sistema
2. Arquitetura	2 min	Estrutura Model / Repository / Service / Controller / DTO
3. Fluxo pelo site	4 min	Ação de candidatura e resultado
4. Comunicação síncrona	4 min	Client HTTP + microserviço + retorno 200
5. Fechamento	2 min	Resumo do fluxo completo

1. Abertura — fala sugerida

> **Fala:** “Boa tarde. Vou apresentar a Entrega 2 da plataforma de estágios. Nesta etapa, além do sistema funcionando, foi organizada a aplicação em camadas, separando Model, Repository, Service, Controller e DTO. Também foi implementada uma comunicação síncrona entre a aplicação principal e um microserviço.”

2. Mostrando o site

Ação: abrir o site que já está no ar e mostrar rapidamente as telas principais. Não precisa explicar cada tela. O objetivo é comprovar que o sistema está funcionando.

> **Fala:** “Aqui temos a aplicação funcionando. Vou usar uma candidatura como exemplo porque ela passa pelo fluxo do backend e também aciona a comunicação com o microserviço.”

3. Arquitetura em camadas

Camada	Função
Model	Representa os dados e o mapeamento das tabelas do banco.
Repository	Centraliza o acesso ao banco e as consultas/CRUD.
Service	Concentra regras de negócio e orquestra o fluxo.
Controller	Recebe a requisição HTTP, valida e retorna a resposta.
DTO	Define o formato dos dados de entrada e saída da API.

Arquivos para mostrar rapidamente

• **vagas/views/api_candidatura.py** — Controller da API de candidatura.
• **vagas/dto/candidatura_dto.py** — Request/Response DTO.
• **vagas/services/candidatura_service.py** — regras e orquestração.
• **vagas/repositories/candidatura_repository.py** — acesso aos dados.
• **vagas/clients/microservico_client.py** — chamada HTTP síncrona.
• **microservico/main.py** — endpoint do microserviço.

4. Demonstração da comunicação síncrona

> **Fala:** “Quando o aluno realiza a candidatura, o Service valida a vaga e chama o microserviço por HTTP. Essa chamada é síncrona: a aplicação espera a resposta do microserviço para continuar o processamento.”

Mostrar no código: no arquivo **vagas/clients/microservico_client.py**, mostrar a chamada **httpx.get(...)** para o endpoint de validação da vaga.

Depois: mostrar o terminal do microserviço com algo como **GET /validar-vaga/1 ... 200 OK**.

> **Fala:** “Esse 200 OK comprova que houve uma comunicação real entre a aplicação principal e o microserviço durante a operação.”

5. Fluxo para explicar no quadro/tela

> **HTTP → Controller → Request DTO → Service → Repository / Microserviço → Response DTO → JSON**

O ponto principal é deixar claro que cada camada possui uma responsabilidade definida, evitando concentrar toda a lógica em uma única view.

6. Fechamento — fala sugerida

> **Fala:** “Com isso, a Entrega 2 atende à separação em camadas e demonstra uma comunicação síncrona real. O sistema continua sendo utilizado pelo site normalmente, mas agora o backend está organizado para separar acesso a dados, regras de negócio, entrada HTTP, contratos e comunicação com o microserviço.”

Checklist antes da apresentação

■ Site funcionando e aberto no navegador

■ Django rodando

■ Microserviço rodando na porta 8004

■ Terminal do microserviço visível para mostrar o GET e o 200 OK

■ Repositório GitHub atualizado com o commit da Entrega 2

■ Ter em mãos os arquivos Controller, Service, DTO, Client e microserviço